



Security Review For Byzantine Finance



Collaborative Audit Prepared For: **Byzantine Finance**
Lead Security Expert(s): **xiaoming90**
Date Audited: **June 1 - June 3, 2026**

Introduction

Byzantine offers a low-risk, institutional-grade digital credit product built for corporate cash. It combines the safety of traditional finance with the efficiency and returns of digital markets.

Scope

Repository: Byzantine-Finance/credit-vault-v2

Audited Commit: e3f982fb0aa2e7a22f2e7aaf3d4765b0cf82d18d

Final Commit: cfcc4b7a123a7279620e15a9515c4b0879d4d1a7

Files:

- src/adapters/ByzantineEurVaultAdapterFactory.sol
- src/adapters/ByzantineEurVaultAdapter.sol

Repository: Byzantine-Finance/fx-hedge-contract

Audited Commit: cd9c93f6e159241857bea4bd25d46a7fb92f5a80

Final Commit: cd9c93f6e159241857bea4bd25d46a7fb92f5a80

Files:

- contracts/ByzantinePrimeEURVault.sol

Repository: Byzantine-Finance/credit-vault-v2

Final Commit: cfcc4b7a123a7279620e15a9515c4b0879d4d1a7

- credit-vault-v2/src/gate/GateBase.sol
- credit-vault-v2/src/gate/GateWhitelist.sol

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
3	1	4

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue H-1: `_clearSettledBatches` re-anchors the in-flight batch's snapshot, causing `realAssets()` to double-count [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-05-byzantine-update-june-1st-2026/issues/4>

Summary

`_clearSettledBatches` re-anchors the in-flight batch's snapshot to the adapter's current balances, which already include that batch's own partially-settled amount, destroying the pre-settlement baseline that `_realAssets` needs. As a result, the batch's still-pending shadow is no longer canceled, and its value is counted twice, so `realAssets()` over-states the adapter's position.

Description

`_realAssets()` avoids double-counting a batch that is mid-settlement by subtracting its settlement progress, measured as `current balance - snapshot taken at batch open`:

```
realAssets (for the in-flight batch) = pendingShadow -  
  ↳ convertToAssets(currentShares - sharesSnapshot)
```

This is only correct while `sharesSnapshotAtBatch` / `eurcSnapshotAtBatch` represent the adapter's balances immediately before this batch began settling. The EUR vault settles deposit/withdraw tickets in chunks and credits `claimableShares` / `claimableEurc` to the adapter before the batch is closed, so the snapshot baseline is what isolates "this batch's own settlement" from everything else.

`_clearSettledBatches` removes settled batches and then re-anchors every remaining open batch to current balances. Because the retention condition keeps batches with `batchId >= currentBatchId`, the currently-settling batch (`batchId == currentBatchId`) is retained and re-anchored. If that batch has already partially settled, its snapshot is overwritten with balances that include its own minted shares, collapsing `current - snapshot` to 0 and silently disabling the correction.

Walkthrough

Scale: 1 EURC = $1e12$ raw `bpEUR` (EURC 6-dp, `bpEUR` 18-dp). Price-per-share fixed at 1.0 (no yield/fees).

Setup

- A1 = 100 EURC deposited via batch1
- A2 = 50 EURC deposited via batch2

- True value the adapter should report at the end = 150 EURC

Timeline of balances

Step	Action	Adapter bpEUR	Notes
1	allocate A1	0	batch1 opens
2	executeDnt	0	batch1 enters settlement
3	allocate A2 (mid-DNT)	0	batch2 opens; snapshot recorded now = (s
4	settle + close batch1	1e20	batch1 mints 100 EURC $\times 1e12 = 1e20$ bpEU
5	executeDnt (batch2)	1e20	batch2 enters settlement
6	processDepositChunk (batch2)	1.5e20	batch2 mints 50 EURC $\times 1e12 = 5e19$ bpEU

The pull fires the re-anchor

- Someone calls `pullClaimableShares()` => `_clearSettledBatches` runs.
- batch1 (settled) is dropped; batch2 survives => re-anchor branch fires.
- It reads `_currentBalances() = (1.5e20, 0)` and overwrites batch2's snapshot:
 - `sharesSnapshot: 0  $\rightarrow$  1.5e20`
 - `eurcSnapshot: 0  $\rightarrow$  0`

Why `realAssets` now double-counts

For the in-flight batch: `realAssets = pendingShadow - convertToAssets(currentShares - sharesSnapshot)`.

- Correct snapshot should be 1e20 (batch1's contribution only, excluding batch2's own mint):
 - `delta = 1.5e20 - 1e20 = 5e19` => `convertToAssets = 50 EURC`
 - batch2 shadow: `50 - 50 = 0` - cancelled
 - Total = 150 bpEUR-value + 0 shadow = 150 EURC (Correct)
- Snapshot the re-anchor actually wrote = 1.5e20 (includes batch2's own mint):
 - `delta = 1.5e20 - 1.5e20 = 0` => `convertToAssets = 0 EURC`
 - batch2 shadow: `50 - 0 = 50` - not cancelled
 - Total = 150 bpEUR-value + 50 leftover shadow = 200 EURC (Wrong)

Result

- Expected: `realAssets = 150 EURC`

- Actual (current code): `realAssets` = 200 EURC
- Over-statement = 50 EURC = exactly A2 - batch2's deposit is counted twice: once as live bpEUR, once as its un-cancelled pending shadow.

Impact

Inflated `realAssets()` misprices VaultV2 shares, leaking value from honest depositors to those who redeem while the overstatement is live.

Recommendation

The current settlement accounting design is complex, and a single current adapter currently mixes all of these into one address:

- settled bpEUR balance
- settled EURC balance
- claimable bpEUR
- claimable EURC
- pending deposits for batch N
- pending withdrawals for batch N
- pending deposits/withdrawals for N+1
- third-party transferred bpEUR/EURC
- DNT-time partially processed tickets

Then `realAssets()` tries to infer settlement progress by comparing

current adapter bpEUR/EURC balance - snapshot taken when batch was opened

Once multiple batches overlap, or a request is made during DNT, or EURC leaves the adapter, or bpEUR is donated, the balance delta no longer uniquely means "this batch settled", and it can mean many things, which leads to different manifestations of edge cases and bugs from that same ambiguity.

Thus, it is recommended to adopt a less complex alternative design. Refer to the Appendix of `realAssets()`, which can understate adapter value when `nextBatchId` is snapshotted during the previous DNT issue, for a potential solution, since they share the same underlying design issues.

Discussion

asanson1404

Fixed in 7c6f82d7 (branch fix/byzantineEurVaultAdapter/minimalCloneProxy).

We adopted the design recommended in the report's appendix: every EUR-vault request is now isolated in its own EIP-1167 minimal-proxy position (`EurVaultPosition`), which is the owner and receiver of its single ticket. The snapshot/delta machinery this finding exploits (`_clearSettledBatches`, snapshots, re-anchoring) has been removed entirely.

Structurally, the double-count can no longer occur: bpEUR minted during a chunked finalize lands on the position's address (not the adapter's), and the position is valued at its stored pending amount until its batch closes – so there are never two overlapping sources of value, and the permissionless sweep only touches positions whose batch has already closed.

Regression test replaying this finding's exact walkthrough (sweep fired during batch N+1's partial settlement; old code reported 200, fixed code reports 150): `testRealAssetsNoDoubleCountAfterSweepDuringPartialSettlement`, commit b813fe7b.

Issue H-2: `realAssets()` can understate adapter value when `nextBatchId` is snapshotted during the previous DNT [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-05-byzantine-update-june-1st-2026/issues/5>

Summary

`ByzantineEurVaultAdapter` tracks open Prime vault batches so `realAssets()` can include deposits and withdrawals that are in-flight. During a Prime vault DNT settlement, `_realAssets()` reduces the pending amount of the batch currently being settled by comparing the adapter's current `bpEUR` and `EURC` balances against the snapshot saved when that batch was opened.

That delta logic is only correct when the snapshot was taken immediately before the same batch began settling. It breaks when batch `N+1` is opened during batch `N`'s DNT. In that path, `N+1` is snapshotted before `N` finishes settling. If `N` then settles and closes, and no state-changing adapter call runs before `N+1` enters DNT, the `N+1` snapshot remains stale.

When `realAssets()` is read during `N+1`'s DNT, the adapter treats batch `N`'s already-counted settlement as if it were settlement progress for batch `N+1`. As a result, the pending value of `N+1` is reduced or zeroed, causing `realAssets()` to temporarily understate the adapter's value.

Description

`_realAssets()` first counts the value already visible to the adapter:

```
total = currentEurc;  
if (currentShares != 0) total += v.convertToAssets(currentShares);
```

It then loops over `openBatchIds` and adds pending deposit and withdraw value for batches that are not yet closed. For the batch currently being settled by the Prime vault, it subtracts the balance delta observed since that batch's snapshot:

```
sharesDelta = currentSharesInt > sharesSnapshot  
    ? uint256(currentSharesInt - sharesSnapshot)  
    : 0;  
pendingDepEurc = pendingDepEurc > v.convertToAssets(sharesDelta)  
    ? pendingDepEurc - v.convertToAssets(sharesDelta)  
    : 0;  
  
eurcDelta = currentEurcInt > eurcSnapshot
```

```
    ? uint256(currentEurcInt - eurcSnapshot)
    : 0;
pendingWithEurc = pendingWithEurc > eurcDelta ? pendingWithEurc - eurcDelta : 0;
```

This is meant to avoid double counting value that has already been minted or paid to the adapter during DNT but before the batch closes. The issue is that the snapshot can represent the wrong baseline.

The stale baseline is created by this sequence:

1. Batch N is opened through an adapter request.
2. The Prime vault enters DNT for batch N.
3. A second adapter request is made during batch N's DNT. Because the Prime vault is not `NormalIdle`, both the adapter and the Prime vault route the request to `nextBatchId`, so the adapter opens batch N+1.
4. The adapter snapshots batch N+1 before batch N has finished settling.
5. The Prime keeper settles and closes batch N. This can mint bpEUR or pay EURC to the adapter, but it does not call the adapter.
6. No state-changing adapter call runs, so `_clearSettledBatches()` does not clear batch N or re-anchor batch N+1.
7. The Prime vault enters DNT for batch N+1.
8. A view read of `realAssets()` uses the stale N+1 snapshot and misattributes batch N's settlement delta to batch N+1.

This is specific to DNT-time `nextBatchId` selection. A batch opened while the Prime vault is `NormalIdle` is snapshotted before its own DNT, so the later delta measures that same batch's settlement. The defect requires a request during batch N's DNT, which opens and snapshots N+1 before N has finished settling.

The re-anchor logic exists in `_clearSettledBatches()`, but it is only reached through state-changing adapter calls such as `allocate`, `deallocate`, `requestDeposit`, `requestWithdraw`, `pullClaimableShares`, or `pullClaimableEurc`. Prime keeper calls and `VaultV2` view reads do not execute it.

Scenario A: deposit side

Assumptions for clean arithmetic:

- `depositFeeBps = 0` and `hedgeSwapFeeBps = 0`.
- bpEUR price per share is 1.00 and frozen during DNT.
- `S=50` means 50e18 bpEUR shares, and 50 EURC means 50e6 EURC units.
- At price per share 1.00, `convertToAssets(S=50) = 50 EURC`.
- The adapter starts with bpEUR value `S=100`, EURC value 0, and no open batches.

Action

0 Initial state

1 Curator allocate(50) routes to batch N

2 Keeper opens batch N DNT

3 Curator allocate(50) during DNT routes to batch N+1, with the snapshot taken before N settles

4 Keeper finalizes N deposit while still in DNT and mints S=50 to the adapter

5 Keeper closes N. No adapter call ran, so no re-anchor occurs

6 Keeper opens batch N+1 DNT

Step 6 arithmetic for Scenario A:

```
currentBatchId = N+1
isDnt = true
currentShares = 150
currentEurc = 0

visible value already counted by _realAssets()
= currentEurc + convertToAssets(currentShares)
= 0 + 150
= 150

batch N contribution
= 0
because N < currentBatchId, so batch N is skipped

batch N settled deposit is already included in the visible value
currentShares = initial shares 100 + batch N settled shares 50
                = 150

batch N+1 pending deposit before delta
= 50

stale delta applied to batch N+1
= convertToAssets(currentShares - sharesSnapshotAtBatch)
= convertToAssets(150 - 100)
= 50

batch N+1 pending deposit after delta
= max(50 - 50, 0)
= 0
```

```

reported realAssets()
= visible value + batch N contribution + adjusted batch N+1 pending deposit
= 150 + 0 + 0
= 150

correct realAssets()
= visible value + genuine batch N+1 pending deposit
= 150 + 50
= 200

understatement
= 200 - 150
= 50

```

The stale `sharesSnapshotAtBatch` should have been re-anchored to 150 after batch N settled. If it had been, the batch N+1 delta would be $150 - 150 = 0$, so its pending deposit would remain 50 and the reported value would be 200.

Scenario B: withdraw side

Assumptions are the same as Scenario A. The adapter starts with bpEUR value $S=200$, EURC value 0, and no open batches. The curator submits two `requestWithdraw(S=50)` calls.

Action

0 Initial state

1 Curator `requestWithdraw(S=50)` routes to batch N and burns $S=50$ immediately

2 Keeper opens batch N DNT

3 Curator `requestWithdraw(S=50)` during DNT routes to batch N+1, with the snapshot taken before

4 Keeper finalizes N withdrawal while still in DNT and pays 50 EURC to the adapter

5 Keeper closes N. No adapter call ran, so no re-anchor occurs

6 Keeper opens batch N+1 DNT

Step 6 arithmetic for Scenario B:

```

currentBatchId = N+1
isDnt = true
currentShares = 100
currentEurc = 50

visible value already counted by _realAssets()

```

```

= currentEurc + convertToAssets(currentShares)
= 50 + 100
= 150

batch N contribution
= 0
because N < currentBatchId, so batch N is skipped

batch N settled withdrawal payout is already included in the visible value
currentEurc = batch N settled EURC payout
              = 50

batch N+1 pending withdraw before delta
= convertToAssets(S=50)
= 50

stale delta applied to batch N+1
= currentEurc - eurcSnapshotAtBatch
= 50 - 0
= 50

batch N+1 pending withdraw after delta
= max(50 - 50, 0)
= 0

reported realAssets()
= visible value + batch N contribution + adjusted batch N+1 pending withdraw
= 150 + 0 + 0
= 150

correct realAssets()
= visible value + genuine batch N+1 pending withdraw value
= 150 + 50
= 200

understatement
= 200 - 150
= 50

```

The stale `eurcSnapshotAtBatch` should have been re-anchored to 50 after batch N paid EURC to the adapter. If it had been, the batch N+1 delta would be $50 - 50 = 0$, so its pending withdraw value would remain 50 and the reported value would be 200.

For a single `realAssets()` read, the understatement is limited to the pending value of the current DNT batch that can be wrongly zeroed. Older batches are skipped, and already-settled bpEUR or EURC remains counted in the adapter's current balances. The error does not make `realAssets()` negative and does not compound across multiple stale batches in one read.

Impact

VaultV2 uses adapter `realAssets()` in `accrueInterestView()` to compute vault total assets and share previews. Since the VaultV2 rate cap limits only upward movement, a lower `realAssets()` value can pass through as an immediate phantom loss.

During the stale-snapshot window, deposits can receive too many shares and redeems can receive too few assets because the vault is priced against understated total assets. When a later state-changing adapter call re-anchors the snapshot and `realAssets()` returns to the correct value, the temporary accounting error can become a permanent value transfer between users.

If performance fees are enabled, the later recovery from the phantom loss can also be treated as positive interest and mint fee shares even though no real economic gain occurred.

Recommendation

The current settlement accounting design is complex, and a single current adapter currently mixes all of these into one address:

- settled bpEUR balance
- settled EURC balance
- claimable bpEUR
- claimable EURC
- pending deposits for batch N
- pending withdrawals for batch N
- pending deposits/withdrawals for $N+1$
- third-party transferred bpEUR/EURC
- DNT-time partially processed tickets

Then `realAssets()` tries to infer settlement progress by comparing

current adapter bpEUR/EURC balance - snapshot taken when batch was opened

Once multiple batches overlap, or a request is made during DNT, or EURC leaves the adapter, or bpEUR is donated, the balance delta no longer uniquely means "this batch settled", and it can mean many things, which leads to different manifestations of edge cases and bugs from that same ambiguity.

Thus, it is recommended to adopt a less complex alternative design. Refer to Appendix A for one possible alternative design for consideration.

Appendix A - Minimal Proxy clones design

A clone is deployed with its own address that handles each deposit or withdrawal request. EIP-1167 Minimal Proxy clones can be used for this.

```
VaultV2
-> ByzantineEurVaultAdapter
  -> DepositPos clone #1
  -> DepositPos clone #2
  -> WithdrawPos clone #1
  -> WithdrawPos clone #2
  -> ...
```

Core idea:

```
One DepositPos = exactly one Prime requestDeposit
One WithdrawPos = exactly one Prime requestWithdraw
```

For deposits, the adapter creates a `DepositPos`, sends EURC to it, and the position submits one deposit request to `ByzantinePrimeEURVault`. The position itself should be the Prime owner/receiver, so any bpEUR or claimable shares are isolated to that position.

For withdrawals, the adapter creates a `WithdrawPos`, sends bpEUR to it, and the position submits one withdrawal request to Prime. Again, the position itself is the owner/receiver, so any EURC or claimable EURC belongs only to that one withdrawal operation.

Valuation becomes simpler:

```
Before the Prime batch closes:
  value the position using the stored pending amount / best-effort value

After the Prime batch closes:
  value the position using actual bpEUR / EURC balances and Prime claimables
```

Settlement detection should be batch-close based:

```
position is settled when Prime.currentBatchId() > position.batchId
```

This no longer relies on adapter-wide balance deltas, which creates the timing-sensitive bugs around DNT, donations, stale snapshots, and overlapping batches.

Once a position's batch is closed, the clone should claim anything claimable from Prime and sweep all bpEUR / EURC back to the main adapter. After sweeping, the position is retired and contributes zero to adapter accounting.

Note 1: This design has a trade-off. This design will lose intra-DNT precision. Example: if `DepositPos A` is finalized early during DNT, it may already have received bpEUR, but we still treat it as pending until the entire batch closes. Same for withdrawals. However, this can be mitigated if `ByzantinePrimeEURVault` is upgradable to expose ticket-level processing info so that each clone could switch from pending valuation to actual

valuation as soon as its exact ticket is processed. If not, a keeper is needed to detect changes and manually update the on-chain valuation method.

Note 2: Valuation involves looping through all positions that have not yet been retired, so there is a risk of an out-of-gas error. However, since VaultV2's allocator/deallocator AND adaptor curator are held by the protocol team, it is expected that they behave non-maliciously and do not spam the system with unnecessary positions. To mitigate this risk, an additional function can be implemented to manually retire or remove settled positions by specifying the batch IDs, rather than looping through all batches to reduce or remove the positions.

Discussion

asanson1404

Fixed in 7c6f82d7 (branch `fix/byzantineEurVaultAdapter/minimalCloneProxy`), by adopting the minimal-proxy position design from this report's appendix.

The stale-baseline problem cannot exist anymore: there are no snapshots to go stale. Settlement detection is close-based per position (`currentBatchId() > position.batchId`) and each position is valued in isolation, so batch N's settlement can never be misattributed to batch N+1 – regardless of whether any adapter call runs between N's close and N+1's DNT.

Regression tests replaying both scenarios of this finding (request made during batch N's DNT, N settles and closes with no adapter call, `realAssets` read during N+1's DNT): `testRealAssetsNoUnderstatementForBatchOpenedDuringPriorDnt` (Scenario A, deposit side) and `testRealAssetsNoUnderstatementForWithdrawOpenedDuringPriorDnt` (Scenario B, withdraw side), commits 08966fa5 and e6a6f171.

Issue H-3: Open-batch EURC snapshot adjustment can fabricate settlement deltas for the next batch and understate VaultV2 totalAssets [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-05-byzantine-update-june-1st-2026/issues/7>

Summary

`requestDeposit()` subtracts the deposited EURC amount from the EURC snapshot of every open batch. This is correct for the currently settling batch, but incorrect for a next batch whose snapshot was taken before the prior batch's EURC settlement arrived.

As a result, the next batch can receive an artificially negative EURC snapshot. When that next batch later enters DNT, `realAssets()` can treat EURC from the prior batch as if it was already received for the next batch's withdrawal settlement, causing the adapter to underreport assets.

Description

The issue occurs when two batches are open during a DNT settlement window.

Example

Assume the following setup:

- 1 bpEUR = 1 EURC
- no fees
- adapter starts with 1,000,000 EURC of exposure

Batch N has a pending withdrawal of 400,000 bpEUR.

current bpEUR value:	600,000
pending withdrawal N:	400,000
eurcSnapshotAtBatch[N]:	0
real exposure:	1,000,000

During DNT for batch N, the adapter requests another withdrawal of 400,000 bpEUR. Since Prime is in DNT, this opens batch N+1.

current bpEUR value:	200,000
pending withdrawal N:	400,000
pending withdrawal N+1:	400,000
eurcSnapshotAtBatch[N+1]:	0

Then batch N settlement sends 400,000 EURC to the adapter.

idle EURC:	400,000
current bpEUR value:	200,000
pending withdrawal N+1:	400,000
real exposure:	1,000,000

Before batch N closes, the curator calls `requestDeposit(400,000 EURC)`. Because Prime is still in DNT, this deposit is queued into batch N+1.

The `requestDeposit()` function will trigger the `_adjustEurcSnapshotOnTransferOut()` function, and the adapter then subtracts 400,000 from all open batch EURC snapshots.

```
File: ByzantineEurVaultAdapter.sol
312:     function _adjustEurcSnapshotOnTransferOut(uint256 amount) internal {
313:         int128 amountInt = int128(int256(amount));
314:         uint256 length = openBatchIds.length;
315:         for (uint256 i; i < length;) {
316:             batchAccounting[openBatchIds[i]].eurcSnapshotAtBatch -= amountInt;
317:             unchecked {
318:                 ++i;
319:             }
320:         }
321:     }
```

For batch N, this is correct because the adapter received EURC for batch N and moved it back out.

For batch N+1, this is incorrect because the batch N+1 snapshot never included that 400,000 EURC.

```
eurcSnapshotAtBatch[N+1]: 0 - 400,000 = -400,000
```

Later, when batch N+1 becomes the current DNT batch, assume no EURC for N+1 has actually arrived yet.

current EURC:	0
current bpEUR value:	200,000
pending deposit N+1:	400,000
pending withdrawal N+1:	400,000
eurcSnapshotAtBatch[N+1]:	-400,000

`realAssets()` calculates the EURC delta as:

```
eurcDelta = currentEurc - eurcSnapshot
eurcDelta = 0 - (-400,000)
eurcDelta = 400,000
```

The adapter then reduces the pending withdrawal value by 400,000, as if the N+1 withdrawal settlement had already arrived.

Reported value:

current bpEUR value:	200,000
pending deposit N+1:	400,000
pending withdrawal N+1:	0
reported realAssets:	600,000

Actual value:

current bpEUR value:	200,000
pending deposit N+1:	400,000
pending withdrawal N+1:	400,000
actual exposure:	1,000,000

The adapter therefore underreports `realAssets()` by 400,000 EURC.

Impact

VaultV2 can read the understated `adapter.realAssets()` during `accrueInterest()` and lock in an artificial loss. This can lower the vault share price and allow new deposits to mint too many shares, diluting existing depositors.

Recommendation

Similar to many other flagged issues, this arises from the design having multiple batches overlap. More specifically, the adapter uses one global EURC balance to infer per-batch settlement, which is unsafe when multiple batches overlap, because EURC received for batch N can still be present when batch N+1 enters DNT.

The best approach is to stop using global adapter balances as the settlement source for all batches. Settlement accounting should be isolated per batch. Refer to the Appendix of `realAssets()`, which can understate adapter value when `nextBatchId` is snapshotted during the previous DNT issue, for a potential solution, since this solution does not overlap multiple batches into a single settlement source, and each settlement is isolated.

Discussion

asanson1404

Fixed in 7c6f82d7 (branch `fix/byzantineEurVaultAdapter/minimalCloneProxy`).

`_adjustEurcSnapshotOnTransferOut` and the per-batch EURC snapshots it corrupted no longer exist. In the new design a mid-DNT `requestDeposit` simply creates a fresh isolated position – there is no shared state to adjust. The finding's setup is additionally impossible by construction: batch N's EURC payout is paid to batch N's own position and stays locked there until the batch closes, so it cannot be redeposited mid-DNT at all.

Regression test replaying the finding's flow (two overlapping withdrawals, batch N settles mid-DNT, deposit requested mid-DNT, `realAssets` read during batch N+1's DNT): `testRealAssetsNoFabricatedDeltaFromMidDntDeposit`, commit `b1c7f29d`.

Issue M-1: Injected bpEUR can be misread as settled deposit tickets causing a temporary VaultV2 asset undercount [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-05-byzantine-update-june-1st-2026/issues/11>

Summary

`ByzantineEurVaultAdapter._realAssets()` uses the adapter's increase in bpEUR balance during `DntInProgress` to infer that the current batch's deposit tickets have settled. This inference is unsafe because the adapter's bpEUR balance can also increase from external sources, such as a direct bpEUR transfer, a third-party claim with `receiver = adapter`, or a Prime vault deposit request using `receiver = adapter`.

As a result, externally injected bpEUR can be mistaken for the adapter's own settled deposit ticket. The adapter then reduces `pendingDepositEurc` even though the deposit is still pending, causing `realAssets()` to temporarily underreport the VaultV2 position.

Description

The adapter snapshots its bpEUR balance when a batch is opened. During DNT, `_realAssets()` compares the current bpEUR balance against that snapshot:

```
uint256 sharesDelta = currentSharesInt > sharesSnapshot ? uint256(currentSharesInt
↪ - sharesSnapshot) : 0;
uint256 eurcDelta = sharesDelta != 0 ? v.convertToAssets(sharesDelta) : 0;
pendingDepEurc = pendingDepEurc > eurcDelta ? pendingDepEurc - eurcDelta : 0;
```

This assumes any increase above the snapshot came from the adapter's own deposit ticket settlement. That assumption is false because `currentShares` includes the adapter's raw bpEUR balance and claimable shares:

```
shares = IERC20(eurVault).balanceOf(address(this)) +
↪ v.claimableShares(address(this));
```

Therefore, any third-party bpEUR sent or minted to the adapter is included in `currentShares`.

Example

Assume 1 bpEUR = 1 EURC.

1. The adapter has `pendingDepositEurc = 100` for the current batch.
2. At batch snapshot, the adapter has 0 bpEUR.

3. Before DNT, a third party sends 100 bpEUR to the adapter.
4. Before DNT starts, the adapter correctly reports:
 - 100 bpEUR held
 - 100 EURC pending deposit
 - `realAssets` = 200
5. When DNT starts, before the adapter's real deposit ticket settles, the adapter sees:
 - `currentShares` = 100
 - `sharesSnapshotAtBatch` = 0
 - `sharesDelta` = 100
6. `_realAssets()` treats this 100 bpEUR delta as settled deposit shares and reduces `pendingDepositEurc` to 0.

The adapter now reports only the injected 100 bpEUR, even though the original 100 EURC deposit is still pending. The true value is still 200, but `realAssets()` reports 100.

This creates a temporary, attacker-triggerable asset drop. Once the real deposit ticket later settles, the adapter balance increases again and the undercount disappears.

Attacker profit example

Assume the same 100 EURC temporary undercount, 1 bpEUR = 1 EURC, no fees, and that the relevant VaultV2 asset increases have already been recognized.

Initial state:

```
Victim owns 1000 VaultV2 shares
VaultV2 totalAssets = 1000
VaultV2 share price = 1.00
Adapter pendingDepositEurc = 100
Adapter sharesSnapshotAtBatch = 0
```

Attack flow:

1. Attacker deposits 9000 EURC.
`totalAssets` = 10000
`totalSupply` = 10000
attacker owns 9000 shares
2. Attacker sends 100 bpEUR to the adapter before DNT.
Adapter value increases from 100 to 200.
`totalAssets` = 10100
share price = 1.01
3. Attacker redeems 9000 shares at 1.01.
attacker receives 9090 EURC
attacker net cash = $-9000 - 100 + 9090 = -10$

4. DNT starts before the real deposit ticket settles.
The injected 100 bpEUR is misread as settled deposit shares.
Adapter value drops from 200 to 100.
totalAssets = 910
totalSupply = 1000
share price = 0.91
5. Attacker deposits 9000 EURC at the depressed price.
attacker receives $9000 / 0.91 = 9890.1$ shares
6. The adapter's real deposit ticket settles.
Adapter value increases from 100 to 200.
totalAssets = 10010
totalSupply = 10890.1
share price = 0.9192
7. Attacker redeems 9890.1 shares.
attacker receives about 9091 EURC

Attacker profit:

```
-9000 initial deposit  
-100 bpEUR injection  
+9090 first redemption  
-9000 second deposit  
+9091 second redemption  
= +81 EURC
```

The attacker does not need to recover the injected bpEUR directly. The profit comes from exiting while the injection is recognized, buying back during the artificial DNT price dip, and redeeming after the real deposit settlement restores the missing value.

Impact

A non-privileged actor can manipulate VaultV2's share price around DNT and extract value from passive shareholders. The extractable amount is bounded by the smaller of the injected bpEUR and the settling batch's pendingDepositEurc, plus available VaultV2 liquidity.

Recommendation

Consider making the adapter donation-resistant. However, since the Prime vault mints directly to the adaptor, it might be challenging to achieve this.

Alternatively, consider another approach to handling settlements from various batches, so that the donation only increases the value of the shares (usually harmless and

unprofitable for the attacker unless coupled with other bugs), but will not screw up the delta/settlement accounting.

Discussion

asanson1404

Fixed in 7c6f82d7 (branch `fix/byzantineEurVaultAdapter/minimalCloneProxy`).

The balance-delta inference this attack relies on has been removed. Pending amounts now live on isolated positions contracts and are never reduced by the adapter's balance: while pending, a position is valued at its stored request amount and reads no balances at all, so injected tokens cannot flip or cancel anything. An injection – to the adapter or to a settled position – can only **add** value, an upward-only direction that the parent vault's `maxRate` cap smooths (per VaultV2's documented donation semantics).

Regression tests: `testRealAssetsInjectedBpEurDoesNotCancelPendingDeposit` (this finding's exact walkthrough – old code dropped to 100, fixed code holds 200), commit a768bbff; plus `testRealAssetsDonationToPendingPositionIsIgnored` and `testRealAssetsDonationToSettledPositionOnlyAddsValue`, commit 08966fa5.

Issue L-1: nextBatchId pending withdrawals are valued with the active DNT's locked PPS [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-05-byzantine-update-june-1st-2026/issues/6>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

When `ByzantinePrimeEURVault` is in `DntInProgress`, new withdrawal requests are correctly queued into `nextBatchId`. However, `ByzantineEurVaultAdapter._realAssets()` values those `nextBatchId` pending withdrawals by calling `ByzantinePrimeEURVault.convertToAssets()`.

During `DntInProgress`, `convertToAssets()` does not return a live PPS and does not compute a price for `nextBatchId`. It uses the active DNT snapshot for the currently settling batch, namely `dntNavEffEurc / dntSupplyPreBatch`.

As a result, a withdrawal that will be priced in a future DNT batch is temporarily included in the adapter's reported assets at the current DNT batch price. This can overstate or understate `VaultV2.totalAssets()` during the active DNT window.

Description

The adapter intentionally mirrors Prime EUR's request batching logic:

```
function _activeBatchId() internal view returns (uint256) {
    IByzantinePrimeEURVault v = IByzantinePrimeEURVault(eurVault);
    return v.vaultState() == IByzantinePrimeEURVault.VaultState.NormalIdle
        ? v.currentBatchId()
        : v.nextBatchId();
}
```

During `DntInProgress`, this returns `nextBatchId`.

`requestWithdraw()` first burns the adapter's `bpEUR` through Prime EUR, then records the burned shares as pending withdrawal shares for the active request batch:

```
IByzantinePrimeEURVault(eurVault).requestWithdraw(
    shares,
    address(this),
    address(this)
);

_adjustSharesSnapshotsOnBurn(shares);
```

```
uint256 batchId = _activeBatchId();

BatchAccounting storage acc = batchAccounting[batchId];
if (!acc.isOpen) {
    _snapshotBalancesForBatch(batchId);
    openBatchIds.push(batchId);
    acc.isOpen = true;
}
acc.pendingWithdrawShares += shares;
```

Prime EUR uses the same request batching rule internally:

```
function _activeBatchIdForRequests() internal view returns (uint256) {
    return vaultState == VaultState.NormalIdle ? currentBatchId : nextBatchId;
}
```

So, while a DNT is active, both contracts agree that the withdrawal belongs to nextBatchId, not to the currently settling batch.

The issue is in how the adapter values that future-batch withdrawal. `_realAssets()` prices every open pending withdrawal with the same global conversion function:

```
uint256 pendingWithShares = acc.pendingWithdrawShares;
uint256 pendingWithEurc = pendingWithShares != 0
    ? v.convertToAssets(pendingWithShares)
    : 0;
```

The only special-case handling is for the batch currently being settled:

```
if (isDnt && batchId == closedBelow) {
    ...
}
```

Here, `closedBelow` is `v.currentBatchId()`. For a withdrawal queued during `DntInProgress`, `batchId` is `nextBatchId`, so `batchId > closedBelow`. The pending withdrawal therefore receives no batch-specific adjustment and is priced directly through `convertToAssets()`.

Wrong Valuation

On the Prime EUR side, `convertToAssets()` is a global PPS conversion:

```
function convertToAssets(uint256 shares) public view override returns (uint256) {
    if (shares == 0) return 0;

    uint256 supply = _conversionSupplyForPps();
    uint256 nav = totalAssets();

    if (supply == 0) {
        return Math.mulDiv(shares, ONE_STABLE, ONE_SHARE);
    }
}
```

```

    }

    return Math.mulDiv(shares, nav, supply);
}

```

During `DntInProgress`, both inputs are pinned to the active DNT snapshot:

```

function _navEffectiveEurc(uint256 hedgedFxEurPerUsd_) internal view returns
↳ (uint256) {
    if (vaultState == VaultState.DntInProgress) {
        return dntNavEffEurc;
    }

    ...
}

function _conversionSupplyForPps() internal view returns (uint256) {
    if (vaultState == VaultState.DntInProgress) {
        return dntSupplyPreBatch;
    }

    return totalSupply() + batches[currentBatchId].totalPendingWithdrawShares;
}

```

Prime EUR also documents the intended semantics:

```

// NOTE: During DntInProgress, conversions reflect the locked snapshot of the
↳ *current* DNT.
// Requests submitted during DNT are queued into nextBatchId and will be priced in
↳ the next DNT.

```

This means a `nextBatchId` withdrawal queued during the current DNT is being marked by the adapter with the current DNT's locked PPS, even though that withdrawal will not settle against the current DNT's pricing snapshot.

The actual future withdrawal settlement uses the DNT snapshot that is active when that future batch becomes `currentBatchId`:

```

(dntNavEffEurc, dntSupplyPreBatch) = _computeBatchPricingInputs(
    hedgedFxEurPerUsd_,
    currentBatchId
);

```

Then settlement computes the gross withdrawal amount from that future snapshot:

```

uint256 grossAssets = Math.mulDiv(t.shares, navEff, supplyPre);

```

The final payout may then be reduced by withdrawal budget haircuts and withdrawal fees:

```

if (haircutModifier < ONE_SHARE) {
    afterSwapAssets = Math.mulDiv(grossAssets, haircutModifier, ONE_SHARE);
}

uint256 feeAssets = 0;
if (feeBps > 0) {
    feeAssets = Math.mulDiv(afterSwapAssets, feeBps, 10_000, Math.Rounding.Ceil);
}

uint256 assetsToSend = afterSwapAssets - feeAssets;

```

So the adapter's interim mark and the eventual settlement are based on different batch snapshots. The adapter is not merely omitting a fee. It is assigning a future-batch withdrawal the PPS of the currently settling batch.

VaultV2 propagation

VaultV2 relies on adapter-reported `realAssets()` when computing total assets:

```

uint256 realAssets = IERC20(asset).balanceOf(address(this));
for (uint256 i = 0; i < adapters.length; i++) {
    realAssets += IAdapter(adapters[i]).realAssets();
}

```

This feeds into `accrueInterestView()` and then into VaultV2 share pricing:

```

uint256 newTotalAssets = MathLib.min(realAssets, maxTotalAssets);
uint256 interest = newTotalAssets.zeroFloorSub(_totalAssets);

```

The same path is used by previews:

```

function previewDeposit(uint256 assets) public view returns (uint256) {
    (uint256 newTotalAssets, uint256 performanceFeeShares, uint256
    ↪ managementFeeShares) =
        accrueInterestView();

    uint256 newTotalSupply = totalSupply + performanceFeeShares +
    ↪ managementFeeShares;
    return assets.mulDivDown(newTotalSupply + virtualShares, newTotalAssets + 1);
}

```

`deposit()`, `mint()`, `withdraw()`, and `redeem()` all call `accrueInterest()` before calculating shares or assets. Therefore, if the adapter reports a stale or wrong value for a `nextBatchId` and pending withdrawal during an active DNT, VaultV2 can use that value for user-facing share minting and burning.

The bad value can also be persisted into `_totalAssets` through `accrueInterest()`:

```

function accrueInterest() public {

```

```

(uint256 newTotalAssets, uint256 performanceFeeShares, uint256
↳ managementFeeShares) =
    accrueInterestView();

emit EventsLib.AccrueInterest(_totalAssets, newTotalAssets,
↳ performanceFeeShares, managementFeeShares);
_totalAssets = newTotalAssets.toUint128();

...
}

```

Upward misstatements are bounded by VaultV2's `maxRate` cap, but downward misstatements can be realized as losses immediately. In either direction, the adapter's stale batch valuation can affect VaultV2 accounting before the future Prime EUR batch is actually priced.

Example

1. Setup during the active DNT: Prime EUR is settling `currentBatchId = 10` and is in `DntInProgress`. The active DNT snapshot implies 1.00 EURC / bpEUR, while `nextBatchId = 11`. At this point, the adapter holds 100 bpEUR.
2. Withdrawal request during the active DNT: The adapter curator calls `requestWithdrawal(100 bpEUR)`. Prime EUR burns the 100 bpEUR immediately and queues the withdrawal into batch 11, because requests made during `DntInProgress` are routed to `nextBatchId`. The adapter mirrors this by recording `pendingWithdrawShares[11] = 100 bpEUR`.
3. Interim adapter valuation: `_realAssets()` values the pending withdrawal through `convertToAssets(100 bpEUR)`. Because Prime EUR is still in `DntInProgress`, `convertToAssets()` uses batch 10's locked DNT snapshot, not a future batch 11 price. Since that snapshot implies 1.00 EURC / bpEUR, the adapter reports the pending withdrawal as 100 EURC.
4. Future batch repricing: Before batch 11 is priced, NAV, FX, hedge margin, USD vault value, withdrawal budget, or batch supply changes. When batch 11 later becomes the active DNT batch, its locked gross PPS is 0.80 EURC / bpEUR. The withdrawal's gross settlement value is therefore about 80 EURC before any withdrawal fees or haircuts, even though VaultV2 may already have accrued or previewed the position at 100 EURC.

The reverse direction is also possible. If batch 11 later prices at 1.20 EURC / bpEUR, the adapter temporarily reports 100 EURC even though the future gross settlement value is about 120 EURC.

Impact

This can either overstate or understate VaultV2 `totalAssets`.

If overstated, VaultV2's share price is too high. Redeemers can receive too much value

per share if idle liquidity or liquidity-adapter liquidity is available. New depositors receive too few shares. The loss from the future Prime EUR settlement can be pushed onto remaining holders once the real value is recognized.

If understated, VaultV2's share price is too low. New depositors can receive too many shares and dilute existing holders. Existing holders can absorb an artificial loss before the future withdrawal batch is actually priced.

Recommendation

The future withdrawal settlement parameters are not knowable at the time the adapter records the pending withdrawal. The eventual value can depend on future NAV, FX, hedge margin, USD vault value, batch supply, queued deposits and withdrawals, withdrawal budget, haircuts, fees, and DNT execution state. Until that future batch becomes the active DNT batch and Prime EUR locks its new snapshot, there is no deterministic price for the adapter to use.

As such, the current valuation is best understood as a best-effort mark, and this should be explicitly documented, so that users are aware of the potential impacts.

Discussion

asanson1404

Acknowledged.

This behavior remains in the redesigned adapter: a pending withdraw position is valued via `convertToAssets(pendingShares)`, which quotes the active DNT's locked PPS during `DntInProgress`, including for positions queued on `nextBatchId`. The EUR vault does not expose a live PPS during a DNT (`totalAssets()` returns the locked `dntNavEffEurc`), so this cannot be improved adapter-side.

The error is bounded by the PPS drift between two consecutive batches (hours), transient (the position is priced exactly at its own DNT), and dwarfed by the accepted settlement-window imprecision already documented. Documented in `EurVaultPosition.value()` NatSpec, commit 39ef3f27.

Issue L-2: Adapter reports pending withdrawals gross of Prime withdraw fees [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-05-byzantine-update-june-1st-2026/issues/8>

Summary

VaultV2 can overstate total assets after the adapter requests a withdrawal from ByzantinePrimeEURVault and before the Prime batch is closed.

The adapter tracks the withdrawal as `pendingWithdrawShares` and values those shares with `eurVault.convertToAssets(...)`, which is a fee-less conversion. However, Prime applies `withdrawFeeBps` later during DNT settlement, so the adapter reports the gross withdrawal value while the amount it will actually receive is net of fees.

Description

When the adapter requests a Prime withdrawal, Prime burns the bpEUR shares immediately:

```
IByzantinePrimeEURVault(eurVault).requestWithdraw(shares, address(this),  
  ↪ address(this));  
acc.pendingWithdrawShares += shares;
```

The adapter then reports those pending shares as assets in `_realAssets()`:

```
uint256 pendingWithEurc = pendingWithShares != 0 ?  
  ↪ v.convertToAssets(pendingWithShares) : 0;
```

This is incorrect because `convertToAssets()` does not include the Prime withdraw fee. The actual Prime settlement path applies the fee later:

```
feeAssets = Math.mulDiv(afterSwapAssets, feeBps, 10_000, Math.Rounding.Ceil);  
assetsToSend = afterSwapAssets - feeAssets;
```

Example

Assume the adapter owns bpEUR worth 1,000 EURC and `withdrawFeeBps = 100`, which is 1%. The adapter curator requests withdrawal for all shares. There is no haircut, FX movement, or swap loss.

Immediately after the request:

```
Adapter pending withdrawal value reported to VaultV2 = 1,000 EURC
```

At Prime settlement:

```
grossAssets = 1,000 EURC
feeAssets    = ceil(1,000 * 100 / 10,000) = 10 EURC
net payout   = 990 EURC
```

VaultV2 therefore sees 1,000 EURC, but the adapter can only recover 990 EURC. The 10 EURC difference is an overstated asset value.

This can also remain visible during partial DNT settlement. If the adapter receives 990 EURC while the batch is not yet closed, `_realAssets()` subtracts only the net EURC delta from the gross pending value:

```
gross pending value = 1,000 EURC
received EURC delta = 990 EURC
remaining pending shadow value = 10 EURC
```

So the adapter can continue reporting the full 1,000 EURC until Prime closes the batch and the pending withdrawal is no longer counted.

Impact

VaultV2 share pricing can be inflated while the Prime withdrawal batch is pending. Users who exit before the batch closes can avoid the fee loss, shifting it to remaining users when the batch finally settles and the loss is reflected.

Recommendation

Consider using Prime's existing fee-aware view function instead of the fee-less conversion when computing the `_realAssets()`.

Within the `_realAssets()`:

```
- uint256 pendingWithEurc = pendingWithShares != 0 ?
  ↳ v.convertToAssets(pendingWithShares) : 0;
+ uint256 pendingWithEurc = pendingWithShares != 0 ?
  ↳ v.previewRedeemNetAssets(pendingWithShares) : 0;
```

This makes pending withdrawal accounting use the same protocol withdraw fee logic as Prime:

```
grossAssets = 1,000 EURC
withdrawFeeBps = 100
fee = 10 EURC
adapter-reported pending value = 990 EURC
```

instead of incorrectly reporting 1,000 EURC.

Important: Do not store the net amount at `requestWithdraw()` time, because Prime applies `withdrawFeeBps` at settlement time and the fee can change before DNT starts. The adapter should compute the pending withdrawal value dynamically inside `_realAssets()` so VaultV2 always uses the current fee schedule.

Discussion

asanson1404

Fixed – we adopted this report's recommendation.

`EurVaultPosition.value()` now values a pending withdraw via `previewRedeemNetAssets(pendingShares)` instead of `convertToAssets(pendingShares)`, deducting the protocol withdraw fee from the moment the shares are burned. The fee is read live at every valuation (not stored at request time), so governance fee changes are tracked automatically, and since the preview computes the fee on the gross amount it never under-states the actual settlement payout.

The over-stated window for the deterministic fee component is eliminated: `realAssets()` recognizes the certain fee cost at request time rather than at batch close. The remaining settlement-window imprecision is the swap-fee / batch-haircut component only (unknowable before DNT execution – see issues #9/#10).

Tests: `testPendingWithdrawValuedNetOfWithdrawFee` (fee recognized at request, payout matches the pending valuation exactly, no step at close) and the updated `testSwapFeeResidueOnWithdrawChunkVanishesAtClose` (residue now bounded by the swap component alone), commit 81add2ca.

Issue L-3: VaultV2 can account Prime EUR exposure at a stale pre-DNT value during an open DNT [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-05-byzantine-update-june-1st-2026/issues/9>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

During `DntInProgress`, Prime pins `convertToAssets()` to the pre-DNT snapshot, `dntNavEffEurc / dntSupplyPreBatch`. This is correct for deterministic current-batch ticket settlement, but the adapter also uses that same conversion as VaultV2's live asset mark. After Prime has locked actuals or processed tickets with fees, haircuts, or under-receipts, but before Prime closes the batch, `adapter.realAssets()` can still report the old/stale gross value instead of the actual settled value.

Description

Prime finalization can be chunked across multiple calls. A finalize call can lock actuals, recognize inbound funds, invest received USDC, and process some deposit or withdrawal tickets while `vaultState` remains `DntInProgress`. In that state, `convertToAssets()` still uses the locked pre-DNT PPS.

The adapter tries to avoid double counting processed current-batch tickets by comparing its current balances against stored snapshots. For deposits, it reduces `pendingDepositEurc` by the value of newly received `bpEUR`. For withdrawals, it reduces pending withdrawal value by the newly received `EURC`. This works only when the ticket settles at the same gross value implied by the locked PPS.

Example:

Assume the adapter requests a 100 `EURC` current-batch deposit. The locked DNT PPS is 1 `bpEUR` = 1 `EURC`. Prime applies a 1% deposit haircut and processes the adapter's ticket before the batch closes.

1. Adapter records the request as `pendingDepositEurc = 100`.
2. Prime processes the ticket with the 1% haircut.
3. Prime mints `bpEUR` worth only 99 `EURC` at the locked PPS.
4. Adapter sees the new `bpEUR` and subtracts 99 from `pendingDepositEurc`.
5. `pendingDepositEurc` becomes 1.

At this point, the adapter's accounting is:

new bpEUR value	99
leftover pendingDepositEurc	1
adapter.realAssets() contribution	100
actual processed-ticket value	99
overstatement	1

The missing 1 EURC was already lost to the haircut, but the adapter still counts it as pending value until Prime closes the DNT.

The issue is not that Prime priced the ticket incorrectly. Prime intentionally settled the ticket at 99 after haircut. The issue is that VaultV2 still receives 100 from `adapter.realAssets()` during the open-DNT window.

The same pattern applies to current-batch withdrawals. If the adapter burns bpEUR worth 100 EURC and Prime settles the withdrawal for only 99 EURC after a fee or haircut, the adapter receives 99 but leaves 1 as pending withdrawal value. `realAssets()` reports 100 until the batch closes.

This is also not limited to the adapter's own current-batch tickets. Already-held bpEUR is still valued through the locked `convertToAssets()` during `DntInProgress`. For example, if the adapter holds 100 bpEUR, locked PPS is 1, and locked actuals imply the post-close PPS should be 0.95, the adapter still reports 100 instead of 95 until Prime exits DNT. Following is an example:

- Adapter holds 100 bpEUR.
- Locked PPS ($\text{dntNavEffEurc} / \text{dntSupplyPreBatch}$) = 1.0, so `convertToAssets(100)` = 100.
- But during the open DNT, Prime has already locked actuals (received funds, haircuts, fees, FX) that imply the real post-close PPS will be 0.95, meaning those 100 shares are truly worth ~95.
- Yet because the vault is still `DntInProgress`, `convertToAssets()` keeps returning 100, not 95.
- So `_realAssets()` reports 100 until Prime exits DNT and the live PPS updates to 0.95, at which point it drops to 95.

Impact

VaultV2 uses `adapter.realAssets()` in `accrueInterest()` before deposits, mints, withdrawals, and redeems. If the adapter is overstated during the open-DNT window, users can withdraw or redeem at an inflated share price using idle or fast-liquidity assets, leaving the later recognized Prime loss to remaining VaultV2 depositors.

Recommendation

The current live mark-to-market valuation approach is a best-effort approach. Efforts can still be taken to deduct known fees and haircuts wherever possible to improve the accuracy. For instance, known haircuts and DNT deductions can be applied once they are available.

```
current deposit pending value -= known deposit haircut
current withdrawal pending value -= known withdrawal haircut and withdraw fee
held bpEUR value -= configured DNT reserve or Prime-provided accounting discount
```

However, such features will require a trustless solution or a keeper and will significantly increase complexity.

Discussion

asanson1404

Acknowledged – the specific mechanism described no longer exists; the bounded residue is documented.

The partial-subtraction path of this finding (the adapter subtracting the post-haircut delta and leaving a phantom remainder) was removed with the snapshot design in 7c6f82d7. In the new design a position is simply valued at its stored pending amount until its batch closes, then at its actual settled holdings – so the residual imprecision is exactly the documented settlement-window over-statement: bounded by realized fees/haircuts, upward-only, and self-clearing at batch close (proven by `testSwapFeeResidueOnDepositChunkVanishesAtClose`, commit 08966fa5, which asserts the residue equals the realized haircut and vanishes exactly at `closeBatch`).

Issue L-4: Adapter overvalues pending Prime EUR batch tickets as batch-level haircut never applied [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-05-byzantine-update-june-1st-2026/issues/10>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`ByzantineEurVaultAdapter.realAssets()` can overvalue pending Byzantine Prime EUR vault deposits and withdrawals during the DNT window because batch-level haircut never applied.

The adapter values its pending tickets using local gross accounting, such as `pendingDepositEurc` and `pendingWithdrawShares`. However, `ByzantinePrimeEURVault` settles tickets using batch-level netting and haircut modifiers.

Description

In `_realAssets()`, the adapter treats pending deposits and withdrawals as if they are standalone claims:

```
pendingDepEurc = acc.pendingDepositEurc;
pendingWithEurc = v.convertToAssets(acc.pendingWithdrawShares);
```

For the active DNT batch, it only subtracts amounts already received into the adapter balance:

```
pendingDepEurc -= convertToAssets(sharesDelta);
pendingWithEurc -= eurcDelta;
```

This is incorrect because Prime vault settlement is not performed per ticket in isolation. Prime first nets the whole batch, then applies batch-wide deposit or withdrawal haircuts. Therefore, the adapter's local pending amount can remain higher than the actual value it will receive.

Example:

```
Adapter pending withdrawal value: 100 EURC
Other users' withdrawals:          9,900 EURC
Total batch withdrawals:           10,000 EURC
Available withdrawal budget:       5,000 EURC

Prime withdrawal haircut = 5,000 / 10,000 = 50%
```

```
Adapter actual payout = 100 * 50% = 50 EURC
Adapter reported value = 100 EURC
Overstatement = 50 EURC
```

The same issue can occur after the adapter's ticket is processed. If the adapter receives only 50 EURC, `_realAssets()` counts the received 50 EURC and still keeps the unmaterialized 50 EURC as pending value. The adapter continues reporting 100 EURC even though the ticket has already settled for only 50 EURC.

This also applies to pending deposits. If a 100 EURC adapter deposit settles with a 99% batch deposit haircut, the adapter receives only 99 EURC worth of bpEUR. However, during the active DNT window, `_realAssets()` still starts from the original `pendingDepositEurc = 100 EURC` and only subtracts the bpEUR value that materialized into the adapter balance.

Example:

```
Adapter pending deposit:      100 EURC
Other users' deposits:       9,900 EURC
Total batch deposits:        10,000 EURC
Batch deposit haircut:       99%

Adapter actual bpEUR value received = 100 * 99% = 99 EURC
Adapter original pending deposit    = 100 EURC
Materialized bpEUR value subtracted  = 99 EURC
Remaining pending deposit reported  = 1 EURC
```

As a result, `_realAssets()` reports both the settled bpEUR value and the leftover pending amount:

```
Current bpEUR value:      99 EURC
Remaining pending deposit: 1 EURC
Total reported value:     100 EURC
Actual settled value:     99 EURC
Overstatement:            1 EURC
```

The 1 EURC remainder is not an additional claim. It is the haircut loss that has already been realized by the Prime vault settlement, but the adapter continues to report it as pending value until the DNT batch is no longer treated as active.

Impact

`VaultV2 accrueInterest()` can use an inflated adapter value when computing total assets, causing Vault V2 interactions to occur at an inflated exchange rate.

Deposit or withdrawal haircuts that have already been realized in Prime can remain temporarily reported as pending value, allowing losses to be shifted to remaining Vault V2 depositors.

Recommendation

For a future batch, or for the current batch before DNT has started, the haircut is not actually known. So, it is not possible to factor in the haircut into the valuation, and valuation is an best-effort mark.

However, the following haircut is already known in the following state and should be included in the valuation approach:

- Active DNT deposit batch - haircut is known once `executeDailyNetTransfer()` has started the DNT and locked the batch snapshot
- Active DNT withdrawal batch when DNT actuals and withdrawal budget are locked

Discussion

asanson1404

Acknowledged – irreducible without modifying the EUR vault.

Batch-level netting and haircut modifiers are only determined at DNT execution (they depend on the whole batch's flows and the available withdrawal budget), so no per-ticket valuation at request time can include them. The redesigned adapter values pending positions gross until their batch closes, at which point each position settles to its actual (post-netting, post-haircut) proceeds.

The direction of the transient error is deliberately upward-only – smoothed by VaultV2's `maxRate` cap – whereas attempting to estimate the haircut would risk under-statements that pass through immediately. Loss scenarios, including the withdraw-budget haircut, are covered by tests in `e6a6f171`, and the limitation is documented in `EurVaultPosition.value()` NatSpec, commit `39ef3f27`.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.